Arbeitsblatt Level 12 · Welt C · Formale Sprachen · Niveau: nur eA (Leistungskurs)

Grammatik trifft Automat

Reguläre Grammatiken und endliche Automaten

Name

Datum

Kernfrage *Wie hängen reguläre Grammatiken und endliche Automaten zusammen?*

Automaten **prüfen** Wörter, Grammatiken **erzeugen** sie. Beim Übersetzer-Duell hast du gesehen: Für manche Sprachen kann man beides ineinander übersetzen – fast mechanisch, wie mit einem Rezept.

Rechtsreguläre Grammatik und Übersetzungsrezept

Eine Grammatik heißt **rechtsregulär** (kurz: regulär), wenn **alle** Regeln eine dieser Formen haben: **A → aB**, **A → a** oder **A → ε** (a Terminalsymbol, B Nichtterminal). Rechts steht also entweder ε oder **genau ein** Terminalsymbol, das von **höchstens einem** Nichtterminal gefolgt wird.

Automat	Grammatik	Automat	Grammatik
Zustand Si	Nichtterminal Si	Si ist Endzustand	Regel Si → ε
Startzustand	Startsymbol	Fehlerzustand	entfällt (keine Regel)
Übergang Si –a→ Sj	Regel Si → aSj	neuer Endzustand Z mit A – a→ Z	Regel A → a

1 Der MähBot

Basis AFB I–II

Ein Rasenmäh-Roboter versteht drei Befehle: **v** (vorwärts fahren) und **d** (drehen) brauchen eine Zahl aus mindestens einer Ziffer, **m** (Mähwerk an/aus) braucht keine. Jeder Befehl endet mit **#**. Alle Ziffern werden durch **z** dargestellt: Σ = {v, d, m, z, #}. Ein Programm besteht aus mindestens einem Befehl, z. B. `vzz#dz#m#`.

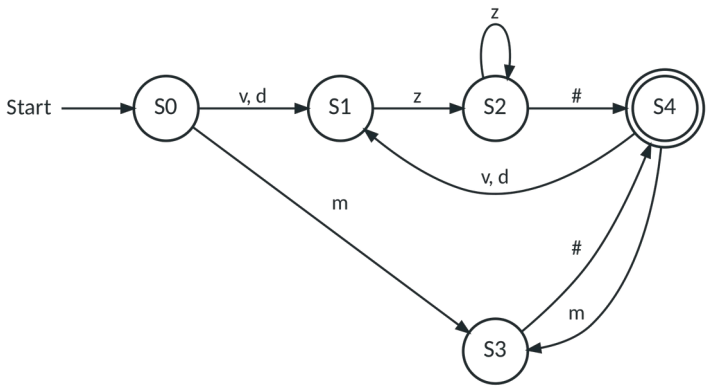


Abbildung 1: DEA für MähBot-Programme. Alle nicht eingezeichneten Übergänge führen in einen Fehlerzustand.

a) **Übersetze** den DEA mit dem Rezept in eine rechtsreguläre Grammatik. **Gib** N, T, Startsymbol und Produktionsregeln **an**.

b) **Zeige** durch Ableiten, dass `vzz#m#` erzeugt wird, und **stelle** die Zustandsfolge des DEA daneben **dar**. Was fällt dir auf?

c) **Erläutere**, warum für den Fehlerzustand keine Regel benötigt wird.

d) *(Profi)* Neu: Wiederholungen wie `wzz(vz#m#)` – nach w folgt die Anzahl (z …), in Klammern stehen Befehle; Wiederholungen sind nicht verschachtelt. **Erweitere** deine Grammatik; sie soll rechtsregulär bleiben. **Beurteile**, ob das auch für beliebig tief verschachtelte Wiederholungen möglich ist.

Kurz erklärt: nichtdeterministischer endlicher Automat (NEA)

- Ein **NEA** darf aus einem Zustand mit demselben Zeichen in **mehrere** Folgezustände wechseln (oder in keinen).
- Ein Wort wird **akzeptiert**, wenn es **mindestens einen Weg** gibt, der nach dem letzten Zeichen in einem Endzustand endet.
- **Teilmengenkonstruktion:** Die Mengen der gleichzeitig möglichen Zustände werden die Zustände eines DEA. Ausführlich: Merkblatt M06 (Bonus-Level 8).

2 Takt für Takt Standard AFB II

Melodien werden ohne Tonhöhen notiert: **v** = Viertelnote (1 Schlag), **h** = halbe Note (2 Schläge), **g** = ganze Note (4 Schläge), **t** = Taktstrich. Jeder Takt hat genau vier Schläge und endet mit t. Die Grammatik G_M erzeugt alle Melodien aus mindestens einem Takt.

N = {S, C, B, A, T}
T = {v, h, g, t}
Startsymbol: S
Produktionsregeln:
S → vC | hB | gT
C → vB | hA
B → vA | hT
A → vT
T → t | tS

Bedeutung der Nichtterminale – im Takt sind noch … Schläge frei:

S	C	B	A	T
4	3	2	1	0 (voll)

Abbildung 2 (links): Grammatik G_M

a) **Gib** zwei Wörter **an**, in denen alle Terminalsymbole vorkommen.

b) **Erläutere** anhand der Grammatik, dass `vhht` kein Wort der Sprache ist.

c) **Begründe**, dass G_M rechtsregulär ist.

d) **Entwickle** mit dem Rezept einen Automaten zu G_M. **Erläutere**, warum er nichtdeterministisch ist, und **entwickle** daraus einen DEA (der Fehlerzustand darf weggelassen werden).

Platz für NEA und DEA

e) **Erläutere**, wie die Grammatik sicherstellt, dass jeder Takt genau vier Schläge hat.

.....

.....

f) *(Profi)* Neu ist die punktierte halbe Note **p** (3 Schläge). **Erweitere** G_M entsprechend.

.....

3 Gäste-WLAN-Codes Standard AFB II–III

Ein Router erzeugt Zugangscodes über Σ = {a, b, 1, 2, #} mit einer regulären Grammatik. Es gilt: **(1)** Im Code kommen mindestens ein **a** und mindestens eine **2** vor (Reihenfolge egal). **(2)** Der Code endet mit **#**; # kommt sonst nicht vor. Gültig sind z. B. `a2#`, `b2a1#`, `2ba#`.

a) **Entwickle** eine rechtsreguläre Grammatik für die Codes. *Tipp: Was muss sich die Grammatik merken – wie die Zustände eines DEA?*

.....

.....

.....

.....

b) **Zeige** mit deiner Grammatik, dass `b2a1#` erzeugt wird.

.....

c) **Gib** eine zusätzliche Regel (4) **an**, sodass die Codes nicht mehr mit einer regulären Grammatik erzeugt werden können, und **begründe** kurz.

.....

.....

4 Grammatik nicht regulär – Sprache aber schon? Profi AFB II–III

N = {S, A}
T = {x, y, +, -}
Startsymbol: S
Produktionsregeln:
S → xA | yA
A → +xA | +yA | -xA | -yA | ε

Abbildung 3: Grammatik G_T für Terme

N = {S}
T = {a, b}
Startsymbol: S
Produktionsregeln:
S → Sa | b

Abbildung 4: Grammatik G_L

a) **Untersuche**, ob x+y-x und x+-y von G_T erzeugt werden.

b) **Begründe**, dass G_T nicht regulär ist, die Sprache $L(G_T)$ aber regulär ist.

c) **Beschreibe** $L(G_L)$, **begründe**, dass G_L nicht rechtsregulär ist, und **gib** eine rechtsreguläre Grammatik für dieselbe Sprache **an**.

d) **Erläutere** am Beispiel von G_T und der Grammatik S → aSb | ab den Unterschied zwischen den Aussagen „Die Grammatik ist nicht regulär.“ und „Die Sprache ist nicht regulär.“

AutomatenLab **Grammatik-Tester (frei)**

Gib deine Grammatik ein: Die App prüft, ob sie rechtsregulär ist, erzeugt Wörter und übersetzt sie in einen NEA/DEA. Ideal zur Kontrolle von Aufgabe 1a, 2d und 3a. Danach: rechtsreguläre Challenges 3, 4, 8 und 9 sowie Challenge 5 (System-Codes) in der Übersicht #/grammatik.

Link: https://automatenlab.frankjuchim.de/#/grammatik?modus=frei

Selbstcheck – Level 12 geschafft, wenn …

- ☐ ich entscheiden kann, ob eine Grammatik rechtsregulär ist.
- ☐ ich einen DEA mit dem Rezept in eine rechtsreguläre Grammatik übersetzen kann – und umgekehrt eine Grammatik in einen NEA/DEA.
- ☐ ich die Bedeutung von Nichtterminalen wie die Bedeutung von Zuständen erläutern kann.
- ☐ ich „Grammatik nicht regulär“ und „Sprache nicht regulär“ unterscheiden kann.